Paper Reading and Summary

ImageNet Classification with Deep Convolutional Neural Networks (AlexNet)

Motivation:

Traditional neural networks were limited in their ability to process large, high-resolution images due to computational constraints and the problem of overfitting with limited training data. The availability of large labeled datasets like ImageNet and increases in GPU computing power created an opportunity to train much larger neural networks. The authors aimed to create a deep convolutional neural network architecture that could effectively classify images into 1000 categories while leveraging these new resources.

Method:

The authors designed a deep convolutional neural network with several key innovations:

1. ReLU (Rectified Linear Unit) activation functions instead of traditional tanh or sigmoid functions to speed up training
2. Multiple GPU training with a specific connectivity pattern between GPUs
3. Local Response Normalization to aid generalization
4. Overlapping pooling to reduce overfitting
5. Data augmentation techniques including image translations, horizontal reflections, and RGB color variations
6. Dropout in fully-connected layers to prevent co-adaptation of neurons

The final architecture consisted of 5 convolutional layers followed by 3 fully-connected layers with a final 1000-way softmax output layer. The network contained 60 million parameters and was trained on 1.2 million high-resolution images from ImageNet using stochastic gradient descent.

Results:

The network achieved top-1 and top-5 error rates of 37.5% and 17.0% respectively on the ILSVRC-2010 test set, significantly outperforming previous state-of-the-art approaches. On ILSVRC-2012, the network achieved a top-5 error rate of 18.2%, and an ensemble of similar networks achieved 15.3%, compared to 26.2% for the second-best entry. The network demonstrated the ability to learn a hierarchy of visual features and showed good generalization capabilities.

Limitations and possible future work:

The network's performance degraded when any convolutional layer was removed, indicating the importance of depth. The authors noted that their approach could benefit from unsupervised pre-training, especially with larger networks and more computational power. They also suggested that applying similar techniques to video sequences could leverage temporal information that is missing in static images. While the network was large by 2012 standards, the authors acknowledged it was still orders of magnitude smaller than the human visual system, suggesting room for growth with more computational resources.

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (Vision Transformer)

Motivation:

Convolutional neural networks (CNNs) have been the dominant architecture in computer vision, while Transformer architectures have revolutionized natural language processing (NLP). However, previous attempts to apply self-attention mechanisms to vision have typically involved combining them with CNNs or using specialized attention patterns. The authors sought to explore whether a pure Transformer architecture, with minimal modifications, could be directly applied to image recognition tasks and compete with state-of-the-art CNNs when trained on sufficient data.

Method:

The Vision Transformer (ViT) treats an image as a sequence of patches, similar to how words (tokens) are processed in NLP Transformers. The approach involves:

1. Splitting an image into fixed-size patches (typically 16×16 pixels)
2. Flattening these patches and linearly projecting them to obtain patch embeddings
3. Adding positional embeddings to retain spatial information
4. Prepending a learnable [class] token to the sequence, whose final state serves as the image representation
5. Processing the resulting sequence through a standard Transformer encoder with multi-headed self-attention

The model was pre-trained on large datasets (ImageNet-21k or JFT-300M) and then fine-tuned on target tasks, often at higher resolution than during pre-training.

Results:

When pre-trained on large datasets like JFT-300M (303M images), ViT outperformed state-of-the-art CNNs on image classification benchmarks while requiring substantially less computational resources to train. The best model (ViT-H/14) achieved 88.55% accuracy on ImageNet, 90.72% on ImageNet-ReaL, and 94.55% on CIFAR-100. The authors found that the performance gap between ViT and ResNets increased with larger datasets, demonstrating that "large scale training trumps inductive bias." With smaller datasets, ResNets performed better, confirming that the convolutional inductive bias is beneficial when training data is limited.

Limitations and possible future work:

The ViT model lacks some of the inductive biases inherent to CNNs, such as translation equivariance and locality, making it less effective when trained on insufficient data. Self-supervised pre-training of ViT showed promising results but still underperformed compared to supervised pre-training, suggesting room for improvement in self-supervised approaches. The authors noted that applying ViT to other computer vision tasks beyond classification, such as detection and segmentation, was an important direction for future work. They also indicated that further scaling of the architecture would likely lead to improved performance.

Deep Residual Learning for Image Recognition (ResNet)

Motivation:

As neural networks became deeper, researchers observed a degradation problem: with increased depth, the accuracy would saturate and then rapidly degrade. Counterintuitively, this performance degradation was not caused by overfitting, as deeper networks exhibited higher training error compared to shallower counterparts. The authors hypothesized that it was becoming increasingly difficult for deeper networks to learn identity mappings when needed. To address this problem, they proposed a residual learning framework to ease the training of very deep networks.

Method:

The key innovation of ResNet is the introduction of residual learning blocks with shortcut connections. Rather than hoping a stack of layers can directly fit a desired mapping H(x), the authors explicitly let these layers fit a residual mapping F(x) = H(x) - x. The original mapping is then recast as F(x) + x, implemented through shortcut connections that perform identity mapping (skipping one or more layers) and element-wise addition.

The authors designed deep residual networks following the VGG-style architecture with two main design principles:

1. For the same feature map size, layers have the same number of filters
2. When the feature map size is halved, the number of filters is doubled to maintain time complexity per layer

The identity shortcuts require no additional parameters and can be directly used when input and output dimensions match. When dimensions increase, they explored two options: zero-padding for identity mapping or using 1×1 convolutions for projection.

Results:

ResNet models demonstrated remarkable success across multiple benchmarks. On ImageNet, the 152-layer ResNet achieved a top-5 error rate of 4.49% (single model) and 3.57% (ensemble), winning the ILSVRC 2015 classification competition. Despite being 8× deeper than VGG nets, ResNet-152 had lower complexity. On CIFAR-10, they successfully trained networks with over 100 layers and showed that very deep residual nets achieved significantly lower error rates than their plain counterparts. The success of ResNet extended beyond classification to detection and localization tasks, where their approach also won in the ILSVRC & COCO 2015 competitions.

Limitations and possible future work:

While ResNet effectively addressed the degradation problem, the authors noted several areas for future exploration. They pointed out that the initialization strategies and normalization methods were still crucial for training very deep networks. They also suggested that the principle of residual learning could be applied to other recognition tasks and non-visual problems. Additionally, the behavior of residual networks raised theoretical questions about why they were more effective than plain networks, particularly regarding the optimization landscape. Finally, while 1000+ layer networks were trainable with their approach, finding the optimal depth for specific tasks remained an open question.

Very Deep Convolutional Networks for Large-Scale Image Recognition (VGGNet)

Motivation:

As deep convolutional networks gained prominence in image recognition tasks, researchers sought to understand how network depth affects accuracy. Previous architectures had used large receptive fields in early convolutional layers, but the optimal network depth and design principles remained unclear. The authors aimed to systematically investigate the impact of depth on performance by creating a series of increasingly deep networks while keeping other architectural choices fixed. Their goal was to determine whether simply adding more layers with small filters could significantly improve performance.

Method:

The key innovation of VGGNet was the use of very small (3×3) convolutional filters throughout the entire network architecture. This design choice offered several advantages:

1. Using stacks of smaller filters instead of larger ones (e.g., three 3×3 layers instead of one 7×7 layer) introduced more non-linearities, making the decision function more discriminative
2. Reduced the number of parameters (three 3×3 layers have $27C^2$ parameters vs. $49C^2$ for one 7×7 layer, where C is the number of channels)
3. Created an implicit regularization by forcing large filters to decompose through smaller ones

The authors created a family of architectures (labeled A through E) with increasing depth from 11 to 19 weight layers. All networks followed the same design principles: small 3×3 filters, preservation of spatial resolution with 1-pixel padding, 2×2 max pooling with stride 2, and three fully-connected layers at the end. For some configurations, they also experimented with 1×1 convolution layers as additional non-linearities.

Results:

The experiments confirmed that depth significantly improves performance. The 19-layer VGG-E network achieved a top-5 error rate of 7.3% on the ImageNet validation set using a single-scale model, which further improved to 6.8% when combining multi-scale and multi-crop evaluation strategies. The VGG team secured 2nd place in the ILSVRC-2014 classification competition. Notably, the authors found that using 1×1 convolutions (model C) performed worse than using 3×3 convolutions throughout (model D), indicating the importance of capturing spatial context. They also demonstrated that deep networks with small filters outperformed shallow networks with larger filters.

Limitations and possible future work:

Despite their excellent performance, VGG networks have a very large number of parameters (up to 144 million), making them computationally expensive to train and deploy. The authors noted that Local Response Normalization, which was used in AlexNet, did not improve performance while increasing memory consumption and computation time. They also observed that their training approach of initializing deeper networks with weights from pre-trained shallower ones could be potentially replaced by better initialization strategies. Future work could focus on reducing the number of parameters while maintaining performance, exploring different types of non-linearities, and applying the architecture to other vision tasks beyond classification.

Going Deeper with Convolutions (GoogLeNet/Inception)

Motivation:

While increasing the depth and width of neural networks was a straightforward way to improve performance, this approach came with significant drawbacks, including increased computational demands and a higher risk of overfitting. The authors aimed to find a more efficient architecture that could achieve state-of-the-art results while maintaining a reasonable computational budget. Instead of simply increasing the network size uniformly, they sought to design a network that utilized computing resources more intelligently and efficiently.

Method:

The key innovation of GoogLeNet is the Inception module, which allows the network to capture features at multiple scales simultaneously while efficiently managing computational resources. Each Inception module consists of parallel paths with:

1. 1×1 convolutions for dimension reduction and feature extraction
2. 3×3 convolutions preceded by 1×1 convolution bottlenecks
3. 5×5 convolutions preceded by 1×1 convolution bottlenecks
4. 3×3 max pooling followed by 1×1 convolutions

These paths are concatenated along the channel dimension to form the output of the module. The use of 1×1 convolutions before larger filters significantly reduces the computational cost by decreasing the number of input channels to the expensive operations. This allowed the authors to build a deep network (22 layers) without excessive computational demands.

The final GoogLeNet architecture included additional features:

- Auxiliary classifiers at intermediate layers to combat vanishing gradients and provide regularization
- Global average pooling instead of fully-connected layers at the end of the network
- ReLU activations throughout the network
- Dropout for regularization

Results:

GoogLeNet achieved state-of-the-art performance on the ILSVRC 2014 classification challenge, with a top-5 error rate of approximately 6.7%. This result was achieved with a model that used 12× fewer parameters than AlexNet while being significantly more accurate. The network also performed exceptionally well on the detection task, demonstrating its generalizability. The authors emphasized that the computational efficiency of the network made it practical for deployment on platforms with limited resources, including mobile devices.

Limitations and possible future work:

While the Inception architecture provided an effective balance between computational efficiency and accuracy, the authors acknowledged that manually designing the network architecture remained challenging. They suggested that the principles behind the Inception architecture could potentially be used to guide automated network architecture search algorithms. The authors also noted that the architectural decisions, while effective, were somewhat speculative and required empirical validation. Future work could include developing automated tools to optimize the network topology, applying similar architectural principles to other domains beyond computer vision, and further improving the efficiency-accuracy trade-off of deep networks.

You Only Look Once: Unified, Real-Time Object Detection (YOLO)

Motivation:

Traditional object detection systems repurposed classification networks to perform detection by applying them to different regions of an image, either using a sliding window approach (like DPM) or region proposals (like R-CNN). These methods were complex, slow, and consisted of multiple separate components that had to be trained independently. The authors aimed to create a simpler, faster, and more unified approach to object detection that could operate in real-time while maintaining competitive accuracy. They wanted to frame object detection as a single regression problem that could be solved with one neural network evaluation.

Method:

YOLO (You Only Look Once) reframes object detection as a regression problem, predicting bounding boxes and class probabilities directly from full images in one evaluation. The approach divides the input image into an S×S grid (S=7 in the original paper). Each grid cell predicts B bounding boxes (B=2 in the original paper), each with five components: x, y, width, height, and confidence. The (x,y) coordinates represent the center of the box relative to the grid cell, while width and height are relative to the whole image. The confidence score reflects the likelihood of an object being present and the accuracy of the predicted box (using IoU).

Each grid cell also predicts C conditional class probabilities for the object it contains. At test time, class-specific confidence scores are computed by multiplying the box confidence by the conditional class probability.

The network architecture consists of 24 convolutional layers followed by 2 fully connected layers. The initial convolutional layers extract features, while the fully connected layers predict the final output probabilities and coordinates. The authors also created a faster version called Fast YOLO with fewer convolutional layers (9 instead of 24) for applications requiring higher speed.

Results:

YOLO achieved impressive results, especially considering its speed. The base model processed images at 45 frames per second (FPS) on a Titan X GPU, while Fast YOLO ran at 155 FPS. On the PASCAL VOC 2007 dataset, YOLO achieved 63.4% mAP (mean Average Precision), and Fast YOLO achieved 52.7% mAP - more than twice as accurate as other real-time detectors at the time. While YOLO's accuracy was lower than state-of-the-art but slower systems like Faster R-CNN (which achieved 73.2% mAP), it made significantly fewer background errors, showing better understanding of the global context in images.

The authors also showed that YOLO could generalize better to new domains, outperforming other methods when trained on natural images and tested on artwork. Additionally, they demonstrated that combining YOLO with Faster R-CNN (using YOLO to rescore Faster R-CNN's detections) yielded improved results by reducing false positives.

Limitations and possible future work:

YOLO had several limitations. It struggled with small objects that appear in groups due to the spatial constraints of having each grid cell predict only two boxes with one class. The relatively coarse features

used for predictions also made it difficult to precisely localize objects. Additionally, the model had difficulty generalizing to objects in unusual aspect ratios or configurations.

The loss function treated errors equally in small and large bounding boxes, which didn't reflect their proportional impact on IoU. This resulted in YOLO's main source of error being incorrect localizations. The authors suggested that future work could focus on improving localization accuracy, better handling of small grouped objects, and finding more effective ways to balance the different components of the loss function.

Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

Motivation:

Object detection systems like R-CNN and Fast R-CNN had significantly improved detection accuracy by using region-based convolutional neural networks, but they still relied on external region proposal methods such as Selective Search. These proposal methods were CPU-based, slow, and created a computational bottleneck in the detection pipeline. The authors aimed to eliminate this bottleneck by creating a unified, end-to-end trainable network that could generate high-quality region proposals and perform detection using shared convolutional features. They sought to make object detection faster while maintaining or improving accuracy.

Method:

The key innovation of Faster R-CNN is the Region Proposal Network (RPN), which shares convolutional features with the detection network. The RPN is a fully convolutional network that simultaneously predicts object bounds and objectness scores at each position. It takes an image of any size as input and outputs a set of rectangular object proposals, each with an objectness score.

The RPN operates by sliding a small network over the convolutional feature map output by the last shared convolutional layer. At each sliding-window location, it predicts multiple region proposals parameterized relative to a set of reference boxes called "anchors." These anchors serve as reference points at multiple scales and aspect ratios, providing a more efficient alternative to image pyramids or filter pyramids for multi-scale detection.

The full Faster R-CNN system consists of:

1. A deep convolutional backbone network (ZF or VGG) for feature extraction
2. The RPN for generating region proposals
3. A Fast R-CNN detector that uses the proposed regions

The authors developed a training algorithm that alternates between optimizing the RPN and the Fast R-CNN detector, allowing them to share convolutional layers and form a unified network.

Results:

Faster R-CNN achieved state-of-the-art object detection accuracy on the PASCAL VOC benchmarks while significantly improving speed. With the VGG-16 model, the system achieved 73.2% mAP on PASCAL VOC 2007 and approximately 70% on VOC 2012, outperforming previous approaches like R-CNN and Fast R-CNN with Selective Search proposals. The method effectively reduced the proposal generation time from

about 2 seconds per image (with CPU-based Selective Search) to just 10 milliseconds, leading to an overall detection speed of 5 frames per second on a GPU.

The authors also showed that the RPN generated higher-quality proposals than methods like Selective Search and EdgeBoxes, particularly at higher IoU thresholds. The RPN was able to learn to propose regions from the data itself, rather than relying on engineered features, making it more adaptable to different domains and datasets. The approach was also successful on the MS COCO dataset and formed the foundation for several winning entries in the ILSVRC and COCO 2015 competitions.

Limitations and possible future work:

While Faster R-CNN substantially improved detection speed compared to previous R-CNN variants, it still wasn't fast enough for real-time applications (typically defined as 30+ FPS). The alternating training procedure, while effective, was somewhat complex and time-consuming. The authors suggested that joint training of the RPN and detection networks could potentially simplify and speed up the training process.

The authors also noted that their approach still used image features of a single scale, which might limit accuracy for detecting objects of vastly different sizes. They suggested that future work could explore multi-scale feature extraction while maintaining computational efficiency. Additionally, they pointed out that the RPN and Fast R-CNN detector could potentially benefit from deeper networks and more expressive features, which was later confirmed by subsequent work using deeper architectures like ResNet.

Mask R-CNN

Motivation:

Instance segmentation, which requires both detecting and segmenting each object instance in an image, is a challenging computer vision task that combines elements from object detection and semantic segmentation. Previous approaches to instance segmentation were often complex, involving multiple stages, or exhibited systematic errors when handling overlapping objects. The authors aimed to develop a simple, flexible, and effective framework for instance segmentation that could build upon the success of Faster R-CNN while maintaining high accuracy and reasonable speed.

Method:

Mask R-CNN extends Faster R-CNN by adding a third branch for predicting segmentation masks in parallel with the existing branches for classification and bounding box regression. The mask branch is a small fully convolutional network (FCN) applied to each Region of Interest (RoI), predicting a binary mask for each class independently. This design decouples mask and class prediction, which the authors found to be crucial for good performance.

A key technical contribution of Mask R-CNN is the introduction of RoIAlign, a layer that replaces the RoIPool operation used in Faster R-CNN. RoIPool performs quantization (rounding) of the floating-point coordinates when extracting features, which causes misalignment between the input image and the extracted features. This misalignment is not critical for classification but severely affects the pixel-level accuracy required for mask prediction. RoIAlign eliminates this quantization by using bilinear interpolation to compute the exact values of input features at regularly sampled locations, preserving spatial precision.

The full Mask R-CNN framework consists of:

1. A backbone architecture (ResNet, ResNeXt, or Feature Pyramid Network) for feature extraction
2. A Region Proposal Network for generating candidate object regions
3. A network head that branches into three outputs: class predictions, bounding box refinements, and binary masks

Results:

Mask R-CNN achieved state-of-the-art results on the COCO instance segmentation benchmark without using complex post-processing or multi-stage pipelines. Using a ResNet-101-FPN backbone, it achieved a mask AP of 35.7% on COCO test-dev, outperforming the previous best approach (FCIS+++) which had a mask AP of 33.6%. The authors demonstrated that their approach did not suffer from the systematic errors on overlapping objects that plagued previous methods like FCIS.

The model also excelled at object detection, matching or exceeding the performance of Faster R-CNN baselines. Despite adding a mask branch, Mask R-CNN maintained efficient inference speeds, running at about 5 FPS with a ResNet-101 backbone. Moreover, the framework proved highly versatile and could be easily adapted to other tasks like human pose estimation, where it also achieved state-of-the-art results.

Ablation studies confirmed the importance of key design decisions: RoIAlign improved mask AP by 10-50% relative to RoIPool, especially under strict localization metrics; using per-class binary masks instead of multi-class softmax was essential for good instance segmentation; and fully convolutional mask prediction outperformed approaches using fully connected layers.

Limitations and possible future work:

While Mask R-CNN significantly advanced the state of instance segmentation, its speed (5 FPS) was still not suitable for real-time applications. The authors noted that the model might benefit from more sophisticated backbone architectures and integration with faster object detection frameworks.

Since the mask branch was designed as a pixel-to-pixel mapping, the resolution of the predicted masks was relatively low (typically 28×28 pixels), which could limit the precision of segmentation boundaries, especially for large objects or fine structures. Future work could explore ways to increase the resolution of mask predictions without significantly increasing computational cost.

The authors also suggested that their framework could be extended to other instance-level recognition tasks beyond segmentation and pose estimation, such as depth estimation or 3D shape prediction. Additionally, they noted the potential for incorporating Mask R-CNN into video understanding systems by adding temporal reasoning.

SSD: Single Shot MultiBox Detector

Motivation:

Traditional object detection frameworks followed a multi-stage approach: generating region proposals, resampling features or pixels for each proposal, and then classifying each proposal with a high-quality classifier. While accurate, these approaches were computationally intensive and too slow for real-time applications, even with high-end hardware. Previous attempts to build faster detectors often came at the cost of significantly decreased accuracy. The authors aimed to create a fast, accurate detector that

eliminated the need for region proposal generation and feature resampling while maintaining competitive accuracy compared to two-stage approaches.

Method:

SSD (Single Shot MultiBox Detector) is a fully convolutional neural network that predicts bounding boxes and class probabilities directly from feature maps in a single forward pass. The key innovations of SSD include:

1. Multi-scale feature maps for detection: Instead of using a single feature map, SSD adds several convolutional feature layers to the end of a base network (VGG-16), with each layer decreasing in size progressively. This allows the network to detect objects at multiple scales.

2. Convolutional predictors for detection: Each feature map (either from the base network or the added layers) has a set of default boxes associated with it. For each default box, the network uses convolutional filters to predict both the offset of the box coordinates and the class scores.

3. Default boxes and aspect ratios: At each feature map location, SSD associates a set of default bounding boxes of different scales and aspect ratios, similar to the anchor boxes in Faster R-CNN. However, SSD applies these default boxes to multiple feature maps of different resolutions, allowing for efficient detection of objects of various sizes.

The training process involves matching these default boxes to ground truth boxes based on jaccard overlap (IoU), and computing a weighted sum of localization loss (for bounding box offset predictions) and confidence loss (for class predictions). SSD also employs techniques like hard negative mining to address the class imbalance problem and extensive data augmentation to improve robustness to various object sizes and shapes.

Results:

SSD achieved impressive results in terms of both accuracy and speed. With a 300×300 input size (SSD300), it achieved 74.3% mAP on the PASCAL VOC 2007 test set while running at 59 frames per second (FPS) on a Nvidia Titan X GPU. This significantly outperformed YOLO (63.4% mAP at 45 FPS) and was comparable to Faster R-CNN (73.2% mAP) while being much faster. With a larger 512×512 input (SSD512), the accuracy improved to 76.8% mAP, surpassing Faster R-CNN.

Experimental analysis showed that SSD performed particularly well on larger objects but struggled with smaller objects, which could be partially mitigated by using a larger input size. The authors also demonstrated that using multiple feature maps for detection at different scales was crucial for good performance, as was the use of default boxes with diverse aspect ratios.

SSD was also evaluated on other datasets including COCO and ILSVRC, showing competitive performance across different benchmarks. The model proved to be a good balance between accuracy and speed, making it suitable for real-time applications.

Limitations and possible future work:

Despite its impressive performance, SSD had some limitations. It still struggled with small objects compared to larger ones, as the deepest (and smallest) feature maps might not have enough information to detect tiny objects. This was partly due to the limited receptive field and stride of the network.

The performance of SSD was heavily dependent on the choice of default box scales and aspect ratios, which required careful tuning for different datasets. The authors noted that automating the design of the optimal tiling of default boxes was an open question for future research.

The data augmentation strategy was crucial for SSD's performance, especially for small objects. Without extensive data augmentation, the accuracy dropped significantly. This highlighted the importance of training techniques beyond just the network architecture.

Future work could focus on improving detection of small objects, exploring different base network architectures, and developing more sophisticated methods for generating default boxes. Additionally, the authors suggested that combining SSD with other approaches like feature pyramids or attention mechanisms could further improve its performance.

DETR: End-to-End Object Detection with Transformers

Motivation:

Traditional object detection systems rely on many hand-designed components like anchor generation, non-maximum suppression (NMS), and complex post-processing steps to convert CNN outputs into final detections. These components encode prior knowledge about the task and make strong assumptions about the detection process. The authors aimed to simplify the detection pipeline by removing these hand-designed components and treating object detection as a direct set prediction problem. They sought to create an end-to-end trainable system that could reason about the relations between objects and the global image context without needing specialized components.

Method:

DEtection TRansformer (DETR) is an encoder-decoder architecture based on transformers, designed to directly predict a set of objects in parallel. The key innovations of DETR include:

1. Bipartite matching loss: DETR uses a set-based global loss that forces unique predictions by performing bipartite matching between predicted and ground-truth objects. This determines the optimal one-to-one assignment between predictions and ground truth, avoiding the need for hand-designed assignment rules or NMS.

2. Transformer encoder-decoder architecture: The encoder processes a CNN-extracted feature map using self-attention to model relationships between all positions. The decoder then takes a fixed small set of learned object queries and, through self-attention and encoder-decoder attention mechanisms, transforms them into output embeddings that are decoded into box coordinates and class labels.

3. Parallel decoding: Unlike many previous set prediction approaches that use autoregressive models, DETR decodes all objects simultaneously, allowing for faster inference and the ability to model relationships between objects directly.

The overall architecture consists of:

1. A CNN backbone (ResNet) for feature extraction
2. A transformer encoder that applies self-attention to the flattened feature map
3. A transformer decoder that processes a set of learned object queries

4. Feed-forward networks that predict class labels and bounding box coordinates for each query

The model is trained end-to-end with a combination of classification loss and bounding box loss (L1 and GIoU) applied to the matched pairs from the bipartite matching.

Results:

DETR achieved comparable results to a highly-optimized Faster R-CNN baseline on the COCO object detection dataset. With a ResNet-50 backbone, DETR reached 42.0% AP, similar to Faster R-CNN's 42.0% AP with the same backbone. With a ResNet-101 backbone and the DC5 variant (dilated C5 stage), DETR achieved 44.9% AP, outperforming the corresponding Faster R-CNN model.

Analysis showed that DETR performed particularly well on large objects, significantly outperforming Faster R-CNN in the APL metric (+7.8%), which the authors attributed to the global reasoning capabilities of the transformer. However, it underperformed on small objects, with a lower APS (-5.5%) compared to Faster R-CNN.

The authors also demonstrated DETR's versatility by extending it to panoptic segmentation, where it outperformed competitive baselines by adding a simple mask prediction head to the pre-trained DETR model.

Limitations and possible future work:

DETR had several limitations. It required significantly longer training schedules than traditional detectors to achieve competitive results. The model's performance on small objects was substantially worse than on large objects, suggesting that improvements in handling multi-scale features would be beneficial.

The fixed set of object queries limited the maximum number of objects that could be detected in an image, although this wasn't a practical issue for most datasets where the number of queries (100) was much larger than the typical number of objects per image.

The authors suggested several directions for future work, including improving performance on small objects, potentially by incorporating ideas from Feature Pyramid Networks (FPN). They also noted that the long training schedule was a drawback and suggested that exploring different training strategies or architectural modifications could help address this issue. Finally, they mentioned that extending DETR to other tasks like instance segmentation (which they partially explored with panoptic segmentation) and video understanding could be promising directions.

CornerNet: Detecting Objects as Paired Keypoints

Motivation:

Traditional object detection methods, particularly one-stage detectors, relied heavily on anchor boxes - predefined boxes of various sizes and aspect ratios that serve as detection candidates. While effective, anchor boxes introduced significant drawbacks: they required a large number of boxes (often more than 40k-100k) to ensure sufficient overlap with ground truth objects, creating an imbalance between positive and negative examples during training and slowing down the process. They also introduced many hyperparameters that were typically set through heuristics. The authors aimed to create a new anchor-free detection approach that could eliminate these issues while maintaining competitive accuracy.

Method:

CornerNet proposes detecting objects as pairs of keypoints - specifically, the top-left and bottom-right corners of the bounding box. The approach uses a single convolutional neural network to predict:

1. A heatmap for top-left corners
2. A heatmap for bottom-right corners
3. Embedding vectors for each detected corner

The embedding vectors help group corners that belong to the same object - the network is trained to produce similar embeddings for corners of the same object. This eliminates the need for anchor boxes entirely.

A key innovation in CornerNet is "corner pooling," a specialized pooling operation designed to better localize corners. Since corners often lack local visual evidence (e.g., a corner might be outside the actual object), corner pooling helps the network find corners by looking horizontally and vertically for object boundaries. For example, to identify a top-left corner, the corner pooling layer looks to the right for the topmost edge and to the bottom for the leftmost edge of an object.

The network architecture is based on an hourglass network backbone with two stacked hourglass modules that capture both global and local features. The network also predicts offsets to adjust the corner locations to improve the precision of the bounding boxes.

Results:

CornerNet achieved 42.2% AP on MS COCO, outperforming all existing one-stage detectors at the time of publication. Ablation studies showed that corner pooling significantly contributed to the performance, improving AP by 1.9%. The authors demonstrated that detecting objects as paired keypoints with embeddings was an effective alternative to anchor-based approaches.

The approach was particularly effective at eliminating the need for the thousands of anchor boxes and complex hyperparameter tuning required by previous detectors. It also removed the need for non-maximum suppression (NMS) as a post-processing step, as the corner grouping via embeddings naturally prevented duplicate detections.

Limitations and possible future work:

While CornerNet outperformed other one-stage detectors, it was computationally expensive, primarily due to the hourglass network backbone. The inference speed was slower than some real-time detectors like YOLO and SSD, making it less suitable for applications requiring real-time performance.

The approach also faced challenges in detecting small objects and objects with occlusion, where identifying and matching corners could be difficult. Additionally, the method relied heavily on accurately detecting both corners of an object - if either corner was missed, the entire object would be missed.

The authors suggested that future work could focus on improving the efficiency of the network, enhancing the detection of small objects, and exploring other keypoint representations beyond just the corners. They also noted that their approach could potentially be extended to other tasks such as instance segmentation and 3D object detection by adding additional keypoints or predictions.

SPPNet: Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition

Motivation:

Traditional convolutional neural networks (CNNs) required fixed-size input images (e.g., 224×224 pixels), which was an artificial constraint that limited their effectiveness. When applied to images of arbitrary sizes, the standard approach was to either crop the input image (potentially losing important visual content) or warp/resize it (causing geometric distortion). Both approaches could harm recognition accuracy, especially when objects appeared at different scales. The authors identified that this fixed-size constraint came only from the fully-connected layers, while the convolutional layers could naturally handle inputs of any size. They aimed to create a network architecture that could accept images of arbitrary sizes and scales while maintaining high performance.

Method:

The authors introduced Spatial Pyramid Pooling (SPP), a technique previously successful in traditional computer vision methods, into the CNN architecture. The key innovations include:

1. The SPP layer is added between the last convolutional layer and the first fully-connected layer. It pools the feature maps using a spatial pyramid with bins at different scales (e.g., 4×4, 2×2, and 1×1 grids), and then concatenates these features to form a fixed-length vector regardless of the input image size.

2. The spatial bins have sizes proportional to the feature map dimensions, so the number of bins remains fixed regardless of the input size. This approach preserves the spatial information while accommodating arbitrary input dimensions.

3. The network, named SPP-net, uses the same convolutional filters on images of different sizes, effectively processing them at their original scales and aspect ratios rather than forcing them to fit a fixed input size.

For training, the authors developed both single-size and multi-size training methods. In multi-size training, they trained the network on images of different sizes (e.g., alternating between 180×180 and 224×224) to increase scale-invariance and reduce overfitting.

Results:

SPP-net showed significant improvements across multiple datasets and tasks:

1. On ImageNet 2012, the authors demonstrated that SPP improved the accuracy of various CNN architectures (ZF-5, Overfeat, etc.) by 1.4-2.3% over their no-SPP counterparts, confirming that the benefits were orthogonal to specific network designs.

2. On Pascal VOC 2007 and Caltech101, SPP-net achieved state-of-the-art classification results using only a single full-image representation without any fine-tuning.

3. For object detection, SPP-net dramatically improved efficiency over the then-leading R-CNN method. While R-CNN computed convolutional features repeatedly for thousands of region proposals, SPP-net computed them only once for the entire image and then applied spatial pyramid pooling on the

feature maps for each region. This made SPP-net 24-102× faster than R-CNN while achieving better or comparable accuracy.

4. In the ILSVRC 2014 competition, the SPP-net-based approaches ranked #2 in object detection and #3 in image classification among all 38 teams.

Limitations and possible future work:

While SPP-net effectively addressed the fixed-size input constraint, it still had limitations. The authors noted that the network's performance could be further improved with deeper and more sophisticated architectures. Training was somewhat complex, requiring switching between different network configurations to simulate variable-sized inputs.

Another limitation was that fine-tuning the network was complicated by the SPP layer, as the gradients needed to be pooled back to appropriate locations in the feature maps. The authors also mentioned that even higher recognition accuracy might be achievable by combining multiple input scales at test time, similar to traditional multi-scale approaches in computer vision.

Future work suggested by the authors included applying SPP to more complex CNN architectures, developing better multi-scale training and testing methods, and extending the approach to other vision tasks beyond classification and detection.